

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Naveen Kumar S	Reg.No.:	192421030
Department:	B Tech IT	Date:	28.08.2026

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I Naveen Kumar S - 192421030 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Naveen Kumar S

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANSWERS :

1. DMA-Based Data Transfer from I/O Device to Memory

Objective

Transfer a large block of data from an I/O device to main memory using **DMA (Direct Memory Access)**. The CPU only initializes the DMA controller and handles the completion interrupt, minimizing CPU involvement.

Pseudocode

ALGORITHM DMA_Data_Transfer

INPUT:

Device_ID

Memory_Address

Data_Size

BEGIN

// Step 1: Initialize DMA controller

DMA_Controller.Reset()

// Step 2: Configure DMA parameters

DMA_Controller.Source ← Device_ID

DMA_Controller.Destination ← Memory_Address

DMA_Controller.Transfer_Size ← Data_Size

DMA_Controller.Mode ← DEVICE_TO_MEMORY

DMA_Controller.Interrupt_Enable ← TRUE

// Step 3: Start DMA transfer

DMA_Controller.Start()

// Step 4: CPU performs other tasks

```

WHILE DMA_Controller.Status ≠ COMPLETE DO
    CPU.Execute_Other_Tasks()
END WHILE

// Step 5: DMA generates interrupt when transfer completes
ON DMA_Interrupt DO
    Status ← DMA_Controller.Get_Status()

    IF Status = SUCCESS THEN
        Clear_DMA_Interrupt()
        Notify_CPU("Transfer completed successfully")
    ELSE
        Handle_DMA_Error()
    END IF
END ON

END

```

Explanation

The CPU first configures the DMA controller with the source device, destination memory address, and transfer size. The DMA controller then transfers data directly between the I/O device and memory without requiring the CPU to move every byte. When the transfer is completed, DMA generates an **interrupt**, and the CPU executes the interrupt service routine. This greatly reduces CPU overhead and is suitable for high-speed devices such as disks and network interfaces.

Main advantage: Minimum CPU involvement and efficient high-speed data transfer.

2. Interrupt Handling System with Vectored Interrupts

Objective

Handle interrupts from multiple devices by assigning each device a **priority and interrupt vector**.
The highest-priority interrupt is serviced first.

Pseudocode

ALGORITHM Vectored_Interrupt_Handler

INPUT:

Interrupts from multiple devices

BEGIN

// Initialize interrupt vector table

Initialize_Interrupt_Vector_Table()

// Assign priority to each device

Set_Priority(Device_A, 1)

Set_Priority(Device_B, 2)

Set_Priority(Device_C, 3)

Set_Priority(Device_D, 4)

ENABLE_INTERRUPTS()

WHILE System_Running = TRUE DO

IF Interrupt_Request_Detected() THEN

// Identify all active interrupts

Active_Interrupts ← Get_Active_Interrupts()

// Select highest-priority interrupt

Highest ← Select_Highest_Priority(Active_Interrupts)

// Obtain interrupt vector

Vector $\leftarrow$ Get_Interrupt_Vector(Highest)

// Save current CPU state

Save_CPU_Context()

// Dispatch corresponding service routine

ISR $\leftarrow$ Interrupt_Vector_Table[Vector]

CALL ISR

// Clear/acknowledge interrupt

Acknowledge_Interrupt(Highest)

// Restore CPU state

Restore_CPU_Context()

END IF

END WHILE

END

Interrupt Service Routine

ISR(Device)

BEGIN

Save_Device_State()

IF Device = Device_A THEN

Service_Device_A()

ELSE IF Device = Device_B THEN

Service_Device_B()

```
ELSE IF Device = Device_C THEN
```

```
    Service_Device_C()
```

```
ELSE
```

```
    Handle_Unknown_Interrupt()
```

```
END IF
```

```
Clear_Device_Interrupt(Device)
```

```
Restore_Device_State()
```

```
END
```

Explanation

In a **vectored interrupt system**, every device has a unique interrupt vector that points to its appropriate ISR. When multiple devices generate interrupts, the interrupt controller determines which device has the highest priority. The CPU saves its current state, jumps directly to the corresponding ISR, services the device, clears the interrupt, and resumes the previous task.

Main advantage: Fast interrupt identification and efficient handling of multiple devices.

3. Dynamic Selection Between Memory-Mapped I/O and I/O-Mapped I/O

Objective

Select the appropriate I/O method based on **latency, bandwidth, CPU architecture, and device requirements**.

Pseudocode

ALGORITHM Select_IO_Mapping

INPUT:

Latency_Requirement

Bandwidth_Requirement

CPU_Architecture

Device_Type

OUTPUT:

Selected_IO_Method

BEGIN

IF CPU_Architecture = "ARM64" THEN

// Memory-mapped I/O is commonly used

Selected_IO_Method ← MEMORY_MAPPED_IO

ELSE

IF Latency_Requirement = HIGH AND

Bandwidth_Requirement = HIGH THEN

Selected_IO_Method ← MEMORY_MAPPED_IO

ELSE IF Latency_Requirement = HIGH THEN

Selected_IO_Method ← MEMORY_MAPPED_IO

ELSE IF Bandwidth_Requirement = HIGH THEN

Selected_IO_Method ← MEMORY_MAPPED_IO

ELSE

Selected_IO_Method $\leftarrow$ IO_MAPPED_IO

END IF

END IF

RETURN Selected_IO_Method

END

Explanation

Memory-mapped I/O (MMIO) assigns device registers to normal memory addresses. The CPU can access these registers using normal load/store instructions. It is commonly preferred when **low latency and high-performance device access** are required.

I/O-mapped I/O (Port-mapped I/O) uses a separate address space for I/O devices and requires special I/O instructions. It is traditionally associated with x86 systems.

Simple decision rule

IF high bandwidth OR low latency is required

THEN use Memory-Mapped I/O

ELSE

use I/O-Mapped I/O

Note: In modern systems, the actual choice is strongly determined by the CPU architecture and operating-system/device design; it is not always freely selectable at runtime.

4. Bus Arbitration for PCIe, USB, and Thunderbolt

Objective

Manage communication requests from multiple devices and determine which device gets access to the available communication resources.

Pseudocode

ALGORITHM Bus_Arbitration

INPUT:

Requests from PCIe, USB, Thunderbolt devices

BEGIN

Initialize_Request_Queue()

Initialize_Device_Priorities()

WHILE System_Running = TRUE DO

 // Collect requests

 Requests ← Get_All_Bus_Requests()

 FOR each Request in Requests DO

 IF Request.Interface = "PCIe" THEN

 Request.Priority ← HIGH

 ELSE IF Request.Interface = "Thunderbolt" THEN

 Request.Priority ← MEDIUM_HIGH

 ELSE IF Request.Interface = "USB" THEN

 Request.Priority ← MEDIUM

 END IF

 Add_To_Request_Queue(Request)

END FOR

// Select request to service

Selected_Request $\leftarrow$ Select_By_Priority(Request_Queue)

IF Selected_Request EXISTS THEN

 Allocate_Communication_Resource(Selected_Request)

 Start_Transfer(Selected_Request)

 WAIT until Transfer_Complete OR Timeout

 IF Transfer_Complete THEN

 Release_Communication_Resource()

 Remove_From_Queue(Selected_Request)

 ELSE

 Handle_Transfer_Error()

 Release_Communication_Resource()

 END IF

END IF

// Prevent starvation

Increase_Waiting_Priority(Request_Queue)

END WHILE

END

Explanation

Bus arbitration decides which device can use shared communication resources when several devices request access simultaneously. The algorithm maintains a request queue and assigns priorities. High-performance interfaces such as PCIe may receive higher priority for latency-sensitive transfers, while USB devices can be scheduled fairly to prevent starvation.

The algorithm can combine **priority scheduling and fairness**. A waiting device can gradually receive increased priority so that low-priority devices are not permanently ignored.

Main goals:

- Efficient bandwidth utilization
 - Low latency
 - Fair resource allocation
 - Prevention of starvation
 - Error and timeout handling
-

5. Hybrid I/O Communication: Polling + DMA with Interrupts

Objective

Use **polling for low-speed devices** and **DMA with interrupts for high-speed devices** to achieve efficient overall I/O performance.

Pseudocode

ALGORITHM Hybrid_IO_Communication

INPUT:

Device_List

BEGIN

FOR each Device in Device_List DO

```
Speed ← Get_Device_Speed(Device)
```

```
Data_Size ← Get_Expected_Data_Size(Device)
```

```
IF Speed = LOW AND Data_Size < SMALL_THRESHOLD THEN
```

```
    // Use polling
```

```
    Device.Mode ← POLLING
```

```
    WHILE Device.Is_Active() DO
```

```
        IF Device.Data_Available() THEN
```

```
            Data ← Device.Read()
```

```
            Process_Data(Data)
```

```
        END IF
```

```
    END WHILE
```

```
ELSE
```

```
    // Use DMA for high-speed devices
```

```
    Device.Mode ← DMA
```

```
    DMA_Controller.Configure(
```

```
        Source = Device,
```

```
        Destination = Memory_Buffer,
```

```
        Size = Data_Size
```

```
    )
```

```
    DMA_Controller.Enable_Interrupt()
```

```
    DMA_Controller.Start()
```

```
    // CPU continues other work
```

CPU.Execute_Other_Tasks()

END IF

END FOR

END

DMA Interrupt Handler

ON DMA_Interrupt(Device)

BEGIN

Status $\leftarrow$ DMA_Controller.Get_Status(Device)

IF Status = SUCCESS THEN

Data $\leftarrow$ Read_From_Memory_Buffer()

Process_Data(Data)

Clear_DMA_Interrupt(Device)

ELSE

Handle_DMA_Error(Device)

END IF

END

Explanation

The hybrid mechanism chooses the communication method according to the device's speed and data volume.

For low-speed devices:

- CPU periodically checks the device status.
- This is called **polling**.
- It is simple and effective when devices generate data infrequently.
- Examples include simple sensors and some keyboards.

For high-speed devices:

- DMA transfers large amounts of data directly between the device and memory.
- The CPU does not handle every byte.
- An interrupt is generated when the transfer finishes.
- Examples include high-speed storage, network interfaces, and multimedia devices.

Overall comparison

I/O Method	Suitable For	CPU Usage	Performance
Polling	Low-speed devices	Higher	Good for small transfers
Interrupt I/O	Medium-speed devices	Moderate	Good
DMA + Interrupt	High-speed devices	Very low	Excellent

In short: The hybrid approach uses **polling where the overhead is acceptable and DMA + interrupts where large, high-speed transfers would otherwise consume too much CPU time**. This provides a good balance between simplicity, CPU utilization, latency, and throughput.